

# Concevoir une API en Javascript

*Support de cours pour la formation Concevoir une API en Javascript de  
Novembre 2024*

*Formateur : Dufrène Valérian*

*"Déclaration d'activité enregistrée sous le numéro 32800232680 auprès du  
préfet de région HAUTS-DE-FRANCE"*

## Déroulé et objectif

La formation "Concevoir une API en Javascript" est découpée en 1 chapitres et est accessible à toute personne ayant acquis des bases en Javascript.

Le programme de formation a été conçu pour vous permettre d'effectuer une nouvelle montée en compétences de manière fluide, par rapport à l'apprentissage des bases.

**Objectif final** : Savoir créer et utiliser une API en Javascript.

À la fin du premier chapitre, vous saurez :

→ **Objectif du chapitre 1** : Découvrir l'environnement de conception d'une API, en Javascript.

→ **Module 1** :

- ◆ Comprendre ce qu'est une API.
- ◆ Différencier les types d'API existantes.
- ◆ **Objectif du Module 1** : Découvrir l'utilité d'une API et son environnement.

## Objectif pédagogique

À la fin de cette formation, les participants devraient être capables de :

- Comprendre ce qu'est une API, comment la concevoir et l'utiliser.
- Utiliser des outils dédiés pour créer une API Javascript.

Ce programme permet de progresser du niveau débutant vers des sujets plus avancés.

## Prérequis

### → Connaissances de base en Javascript :

- ◆ Comprendre la syntaxe de Javascript.
- ◆ Savoir créer et utiliser des fonctions, des classes et des méthodes en Javascript.
- ◆ Connaissance des notions de base de la programmation orientée objet (POO), comme les objets, les classes, et les méthodes.

### → Familiarité avec Node & NPM :

- ◆ Connaître les bases de Node pour installer et gérer des bibliothèques avec NPM.

### → Environnement de développement installé :

- ◆ Avoir installé un éditeur de code (comme VS Code, PHPStorm, ou Sublime Text) et savoir l'utiliser pour écrire et exécuter du code.

### → Notions de base sur la ligne de commande :

- ◆ Connaître les commandes de base pour naviguer dans le terminal et exécuter des scripts Javascript.
- ◆ Savoir comment lancer des commandes de NPM et Node via la ligne de commande.

# Chapitre 1 - Initialiser un nouveau projet et créer une structure de données

*Support de cours pour la formation Concevoir une API en Javascript de  
Novembre 2024*

*Formateur : Dufrène Valérian*

*"Déclaration d'activité enregistrée sous le numéro 32800232680 auprès du  
préfet de région HAUTS-DE-FRANCE"*

## Initialisation du projet

Nous allons créer un nouveau projet, qui contiendra le code nécessaire à la création de notre première API.

Avant tout, nous allons avoir besoin de définir quelques points, pour initialiser ce projet et créer une structure de données initiale.

Pour ce projet, nous allons travailler sur une API permettant la création d'un CV en ligne.

→ **Vérifier la version actuelle de Node et NPM :**

◆ **node -v**

◆ **npm -v**

Nous allons utiliser **npm** pour gérer les dépendances de notre projet.

Si vous avez un doute sur la version actuelle de Node, rendez-vous sur <https://nodejs.org/fr>.

Ouvrez un terminal de commande et créez un nouveau répertoire pour le projet.

**mkdir cv-api**

Déplacez-vous dans le dossier créé.

**cd cv-api**

Initialisez le nouveau projet avec npm.

**npm init -y**

Installez la dépendance Express.

**npm install express**

Installez la dépendance PostgreSQL.

**npm install pg**

Créez un fichier **index.js** et ajoutez ce code à l'intérieur.

```
const express = require('express');

const { Pool } = require('pg');

const app = express();

const port = process.env.PORT || 3000;

// Connexion à PostgreSQL
const pool = new Pool({
  user: 'votre_utilisateur',
  host: 'votre_hôte',
  database: 'votre_base_de_données',
  password: 'votre_mot_de_passe',
  port: 5432, // Port par défaut de PostgreSQL
});

// Création de la table utilisateur (à exécuter une fois)
pool.query(`CREATE TABLE IF NOT EXISTS users (
  id SERIAL PRIMARY KEY,
  name TEXT,
  email TEXT,
  resume JSONB
)`)
.then(() => console.log('Table users créée ou déjà existante'))
.catch(err => console.error('Erreur lors de la création de la table:', err));

// Définir les routes (exemple)
app.get('/', (req, res) => { res.send('Hello from your CV API!'); });
app.post('/users', async (req, res) => {
  try {
    const { name, email, resume } = req.body;
    const result = await pool.query('INSERT INTO users (name, email, resume)
VALUES ($1, $2, $3) RETURNING *', [name, email, resume]);
    res.status(201).json(result.rows[0]);
  }
});
```

```
catch (err) {  
  console.error(err);  
  res.status(500).json({  
    message: 'Erreur lors de la création de l'utilisateur'  
  });  
} };  
app.listen(port, () => { console.log(`Server listening on port ${port}`); });
```

### Explications des modifications:

- **Installation de `pg`:** Le module `pg` est utilisé pour se connecter à PostgreSQL.
- **Configuration de la pool:** Les informations de connexion à votre base de données PostgreSQL sont spécifiées dans les options de la pool.
- **Création de la table:** Une requête SQL est exécutée pour créer la table `users`. Le type `JSONB` est utilisé pour stocker le CV au format JSON.
- **Requêtes SQL:** Les interactions avec la base de données se font à l'aide de requêtes SQL exécutées avec la méthode `query` de la pool.
- **Paramètres:** Les paramètres des requêtes sont passés en tableau pour éviter les injections SQL.

### Points clés à noter:

- **Type JSONB:** PostgreSQL offre un type de données JSONB qui est particulièrement adapté pour stocker des données structurées comme des CV.
- **Transactions:** Pour des opérations plus complexes impliquant plusieurs mises à jour, il est recommandé d'utiliser des transactions.
- **Optimisation:** Pour de meilleures performances, il est important d'utiliser des index sur les colonnes fréquemment utilisées dans les requêtes.
- **Sécurité:** Comme pour MongoDB, il est essentiel de sécuriser votre application en utilisant des mécanismes d'authentification et d'autorisation appropriés.

### Avantages de PostgreSQL par rapport à MongoDB:

- **Transactions ACID:** PostgreSQL supporte les transactions ACID (Atomicity, Consistency, Isolation, and Durability), ce qui garantit la cohérence des données.
- **SQL:** PostgreSQL utilise le langage SQL, qui est largement utilisé et bien documenté.

- **Types de données riches:** PostgreSQL offre une grande variété de types de données, y compris les types numériques, les dates, les géométriques et les JSON.
- **Maturité:** PostgreSQL est un système de gestion de base de données relationnelle mature et stable.

**Choisir entre MongoDB et PostgreSQL dépendra de vos besoins spécifiques:**

- **MongoDB** est bien adapté aux applications avec des données schématisées de manière flexible et des besoins en termes de haute disponibilité.
- **PostgreSQL** est un excellent choix pour les applications nécessitant des transactions ACID, des relations complexes entre les données et une grande variété de types de données.

Créez un fichier **.env**, à la racine de votre projet et ajoutez les instructions suivantes.

```
DATABASE_USER=votre_utilisateur
DATABASE_HOST=votre_hôte
DATABASE_NAME=votre_base_de_données
DATABASE_PASSWORD=votre_mot_de_passe
DATABASE_PORT=5432

PORT=3000
```

Installez le package **dotenv**.

```
npm install dotenv
```

Ce package permet de charger les variables d'environnement depuis un fichier **.env** dans le processus Node.js. Installez-le en exécutant la commande suivante dans votre terminal :

Chargez la configuration d'environnement dans votre **index.js**.

```
require('dotenv').config();
```

Vous pouvez maintenant accéder aux valeurs des variables d'environnement à l'aide de



**process.env**. Modifiez la configuration de votre pool de la manière suivante :

```
const pool = new Pool({
  user: process.env.DATABASE_USER,
  host: process.env.DATABASE_HOST,
  database: process.env.DATABASE_NAME,
  password: process.env.DATABASE_PASSWORD,
  port: process.env.DATABASE_PORT
});
```

Vous devriez désormais avoir le code suivant, au début de votre fichier **index.js**.

```
require('dotenv').config();
const { Pool } = require('pg');
const pool = new Pool({
  user: process.env.DATABASE_USER,
  host: process.env.DATABASE_HOST,
  database: process.env.DATABASE_NAME,
  password: process.env.DATABASE_PASSWORD,
  port: process.env.DATABASE_PORT
});
```

### **Pourquoi utiliser un fichier d'environnement ?**

- Sécurité: Les informations sensibles ne sont pas exposées dans le code source.
- Flexibilité: Vous pouvez facilement changer les valeurs des variables d'environnement sans modifier le code.
- Environnements multiples: Vous pouvez avoir différents fichiers **.env** pour différents environnements (développement, production, etc.).

### **Conseils supplémentaires:**

- Ignorer le fichier `.env` dans Git: Ajoutez `.env` à votre fichier `.gitignore` pour éviter de commettre accidentellement les informations sensibles dans votre dépôt Git.
- Variables d'environnement système: Vous pouvez également définir des variables d'environnement au niveau du système d'exploitation, mais il est généralement préférable d'utiliser un fichier `.env` pour une meilleure organisation.
- Autres bibliothèques: Il existe d'autres bibliothèques qui peuvent être utilisées pour gérer les variables d'environnement, mais `dotenv` est l'une des plus populaires.

## Installation des outils et création de la base de données

Pour notre API, nous allons avoir besoin d'utiliser une base de données.

Pour répondre à ce besoin, nous allons utiliser **PostgreSQL**, qui est un outil permettant la création et la gestion de bases de données par terminal et l'outil **PgAdmin**, qui nous permettra de visualiser notre base de données sur une interface graphique.

**Installez PostgreSQL** : <https://www.postgresql.org/download/>.

Lors de l'installation de PostgreSQL, vous aurez une étape de **création de mot de passe**.

Cette étape est obligatoire et le mot de passe défini vous permettra de connecter le compte d'administration **postgres** sur l'outil **PgAdmin**.

Une fois installé, vous devrez ajouter la variable d'environnement **Path**, contenant le chemin vers les dossiers **bin** et le dossier **data** de **PostgreSQL**.

Par défaut, ces chemins sont: **C:\Program Files\PostgreSQL\17\bin** (le 17 correspond à la version installée) et **C:\Program Files\PostgreSQL\17\data**.

Paramètres

Valérien Dufrène  
valerien.dufrene@gmail.com

Rechercher un paramètre

- Accueil
- Système
- Bluetooth et appareils
- Réseau et Internet
- Personnalisation
- Applications
- Comptes
- Heure et langue
- Jeux
- Accessibilité
- Confidentialité et sécurité
- Windows Update

## Système > Informations système

LAPTOP-3A7ALKFQ  
TUF Gaming FX505DT\_FX505DT

Renommer ce PC

### 1 Spécifications de l'appareil

Copier

|                            |                                                        |
|----------------------------|--------------------------------------------------------|
| Nom de l'appareil          | LAPTOP-3A7ALKFQ                                        |
| Processeur                 | AMD Ryzen 5 3550H with Radeon Vega Mobile Gfx 2.10 GHz |
| Mémoire RAM installée      | 32.0 Go (29.9 Go utilisable)                           |
| ID de périphérique         | E66040AA-9ABE-4977-9DAE-5E31772EA963                   |
| ID de produit              | 00325-81803-63567-AAOEM                                |
| Type du système            | Système d'exploitation 64 bits, processeur x64         |
| Stylet et fonction tactile | Prise en charge du stylet                              |

Liens connexes

Domaine ou groupe de travail

Protection du système

Paramètres avancés du système

### Spécifications de Windows

Copier

|                                                 |                                                 |
|-------------------------------------------------|-------------------------------------------------|
| Édition                                         | Windows 11 Famille                              |
| Version                                         | 24H2                                            |
| Installé le                                     | 17/10/2024                                      |
| Build du système d'exploitation                 | 26100.2314                                      |
| Expérience                                      | Windows Feature Experience Pack 1000.26100.32.0 |
| Contrat de services Microsoft                   |                                                 |
| Termes du contrat de licence logiciel Microsoft |                                                 |

Support

Copier

Paramètres

Valérien Dufrène  
valerien.dufrene@gmail.com

Rechercher un paramètre

- Accueil
- Système
- Bluetooth et appareils
- Réseau et Internet
- Personnalisation
- Applications
- Comptes
- Heure et langue
- Jeux
- Accessibilité
- Confidentialité et sécurité
- Windows Update

## Système

LAPTOP-3A7ALKFQ  
TUF Gaming FX505DT\_FX505DT

Renommer ce PC

### 1 Spécifications de l'appareil

Copier

|                            |                                                        |
|----------------------------|--------------------------------------------------------|
| Nom de l'appareil          | LAPTOP-3A7ALKFQ                                        |
| Processeur                 | AMD Ryzen 5 3550H with Radeon Vega Mobile Gfx 2.10 GHz |
| Mémoire RAM installée      | 32.0 Go (29.9 Go utilisable)                           |
| ID de périphérique         | E66040AA-9ABE-4977-9DAE-5E31772EA963                   |
| ID de produit              | 00325-81803-63567-AAOEM                                |
| Type du système            | Système d'exploitation 64 bits, processeur x64         |
| Stylet et fonction tactile | Prise en charge du stylet                              |

Liens connexes

Domaine ou groupe de travail

Protection du système

Paramètres avancés du système

### Spécifications de Windows

Copier

|                                                 |                                                 |
|-------------------------------------------------|-------------------------------------------------|
| Édition                                         | Windows 11 Famille                              |
| Version                                         | 24H2                                            |
| Installé le                                     | 17/10/2024                                      |
| Build du système d'exploitation                 | 26100.2314                                      |
| Expérience                                      | Windows Feature Experience Pack 1000.26100.32.0 |
| Contrat de services Microsoft                   |                                                 |
| Termes du contrat de licence logiciel Microsoft |                                                 |

Support

Copier

Propriétés système

Nom de l'ordinateur

Paramètres système avancés

Protection du système

Matériel

Utilisation à distance

Vous devez ouvrir une session d'administrateur pour effectuer la plupart de ces modifications.

Performances

Effets visuels, planification du processeur, utilisation de la mémoire et mémoire virtuelle

Paramètres...

Profil des utilisateurs

Paramètres du Bureau liés à votre connexion

Paramètres...

Démarrage et récupération

Informations de démarrage du système, de défaillance du système et de débogage

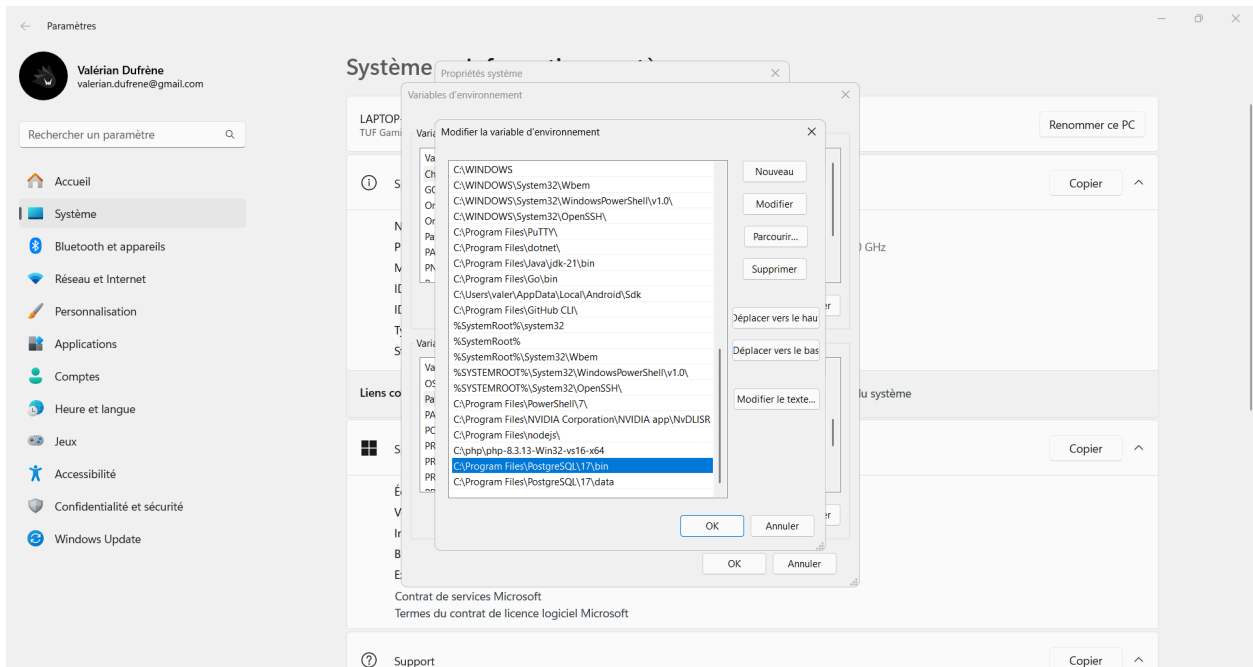
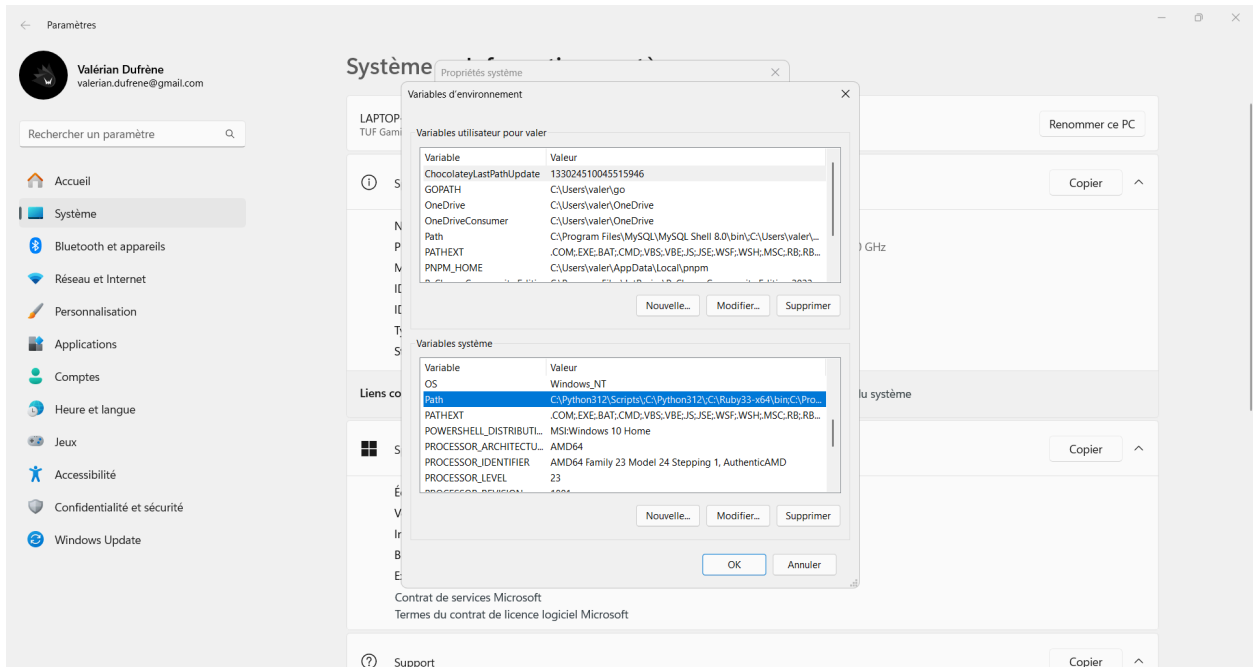
Paramètres...

Variables d'environnement...

OK

Annuler

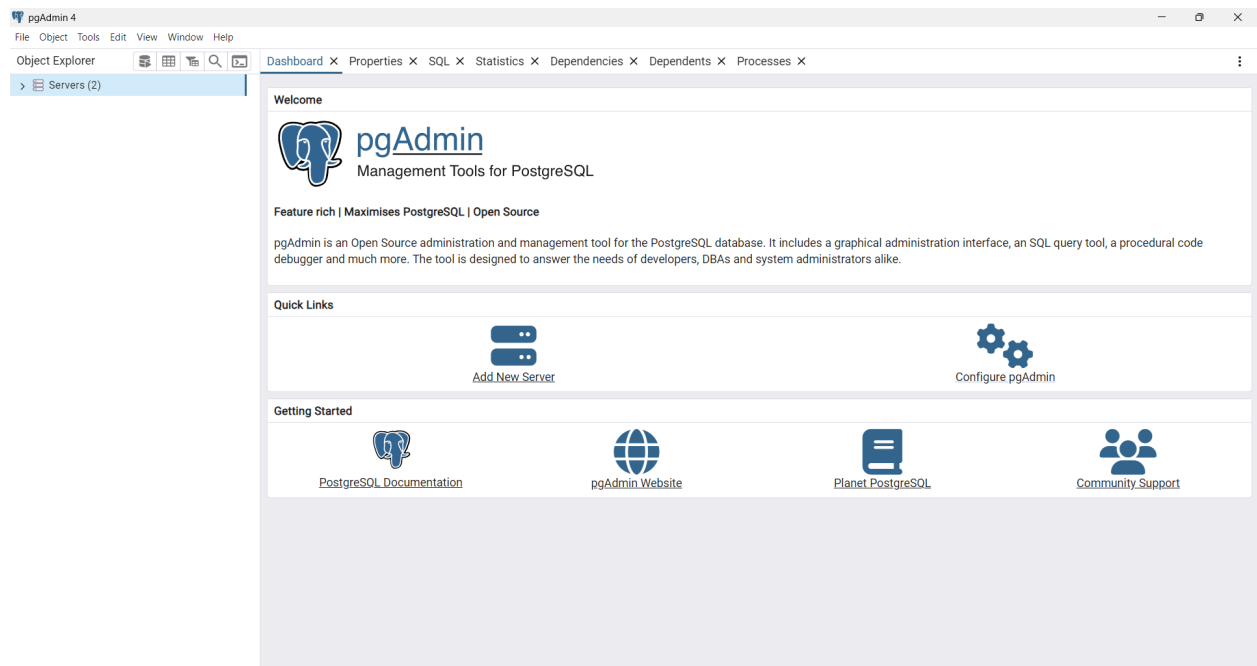
Appliquer



Installez PgAdmin : <https://www.pgadmin.org/download/>.

Lancez PgAdmin.

Sur l'écran d'accueil, cliquez sur **Add New Server**.



Pour notre API, nous avons besoin d'une base de données classique, sans configurations avancées.

Nous allons donc suivre cette configuration pour configurer notre base minimale.

Register - Server

General

Connection

Parameters

SSH Tunnel

Advanced

Name

cv-api

Server group

Servers

Background

Foreground

Connect now?

Comments

Either Host name or Service must be specified.

Close  Reset  Save

## ONGLET GÉNÉRAL

**Name :** Vous permet de donner un nom à votre base de données.

**Connect now ? :** Vous permet une connexion immédiate après l'enregistrement des configurations.

Register - Server

X

General

Connection

Parameters

SSH Tunnel

Advanced

Host name/address

localhost

Port

5432

Maintenance database

postgres

Username

postgres

Kerberos authentication?

☐

Password

....

Save password?

☒

Role

Service

i

?

X Close

Reset

Save

## ONGLET CONNECTION

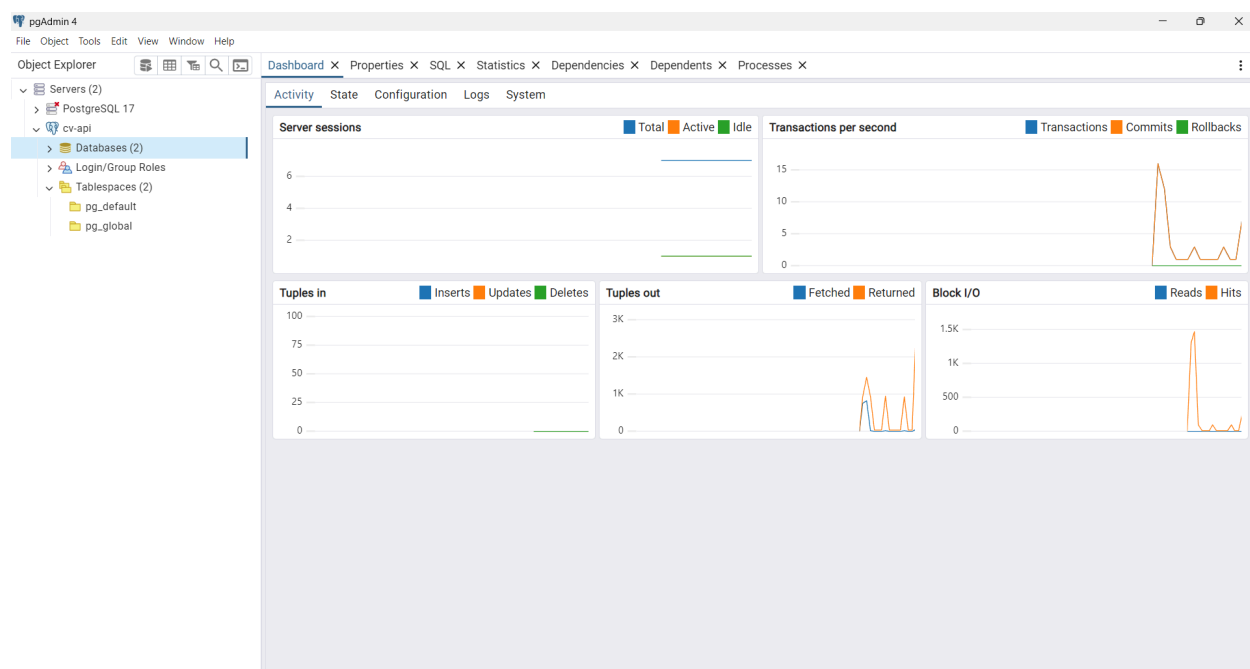
**Host name/adress** : Nous permet de préciser l'hôte/l'adresse permettant de cibler la base de données. Pour notre configuration, on pourra utiliser **localhost** ou **127.0.0.1**.

**Port** : Nous permet de spécifier le port utilisé par la base de données. Par défaut, le port est **5432**.

**Username** : Nous permet d'indiquer le pseudo de l'utilisateur postgre ayant les droits sur la base. Par défaut, il s'agit de l'utilisateur **postgres**.

**Password** : Nous permet d'indiquer le mot de passe de l'utilisateur renseigné précédemment.

Il peut s'agir du mot de passe défini sur l'utilisateur créé lors de l'installation de **PostgreSQL** (si vous utilisez cet utilisateur) ou le mot de passe par défaut est **postgres**.



Pour visualiser vos tables, après leur création, il vous suffit de vous rendre dans :

- **cv-api**
- **Databases**
- **cv-api**
- **Schemas**
- **public**
- **Tables**

Pour visualiser les données d'une table (par exemple, **users**), faites un **clic-droit** sur le nom de la table et sélectionnez l'option **View/Edit datas > All Rows**.

Vous pouvez également cliquer sur le bouton **All Rows** ou utiliser la macro **ALT + SHIFT + V**.



## Utiliser un middleware pour parser les données des requêtes entrantes

Pour ce projet, nous allons utiliser un **middleware** pour parser les données entrées au format **JSON**.

## Compréhension du middleware `express.json()`

### Qu'est-ce qu'un middleware ?

Dans le contexte d'Express.js, un **middleware** est une fonction qui intercepte les requêtes HTTP avant qu'elles n'atteignent le routeur. Il permet d'effectuer des traitements sur ces requêtes, comme :

- **L'authentification:** vérifier si l'utilisateur est autorisé à accéder à une ressource.
- **La validation:** s'assurer que les données envoyées par le client sont conformes à un certain format.
- **La journalisation:** enregistrer les requêtes dans un fichier log.
- **Le parsing:** transformer les données brutes de la requête en un format utilisable par l'application.

### Le rôle de `express.json()`

Le middleware `express.json()` est spécifiquement conçu pour **parser** les données au format **JSON** envoyées dans le corps d'une requête HTTP. En d'autres termes, il prend les données brutes au format texte (qui représentent un objet JavaScript) et les convertit en un véritable objet JavaScript que vous pouvez manipuler directement dans votre code.

### Pourquoi est-ce utile ?

- **Simplification du traitement des données:** Au lieu de devoir parser manuellement les données JSON, vous pouvez directement accéder aux propriétés de l'objet JavaScript créé par le middleware.
- **Sécurité:** Le middleware `express.json()` vous aide à sécuriser votre application en vous permettant de valider les données reçues avant de les traiter.
- **Interopérabilité:** Le format JSON est très largement utilisé pour l'échange de

données entre différentes applications, ce qui rend `express.json()` particulièrement utile.

## Un exemple concret

Imaginez une application qui permet aux utilisateurs de créer de nouveaux utilisateurs. Un utilisateur envoie une requête POST à l'API avec les informations du nouvel utilisateur au format JSON dans le corps de la requête.

**// Requête POST envoyée par le client**

```
{  
  
  "name": "John Doe",  
  
  "email": "johndoe@example.com"  
}
```

Grâce à `express.json()`, vous pouvez accéder à ces données directement dans votre routeur :

```
app.post('/users', (req, res) => {  
  
  const newUser = req.body; // newUser est un objet JavaScript  
  
  console.log(newUser.name); // Affiche "John Doe"  
  
  // ... Enregistrer le nouvel utilisateur dans la base de données ...  
  
});
```

## En résumé

Le middleware `express.json()` est un outil essentiel dans le développement

d'applications Express.js qui utilisent le format JSON. Il simplifie considérablement la gestion des données reçues par votre serveur, en vous permettant de vous concentrer sur la logique métier de votre application.

En ajoutant `app.use(express.json());` au début de votre application Express, vous indiquez à Express que vous souhaitez parser toutes les requêtes qui contiennent des données JSON dans leur corps.

Pour aller plus loin:

- **express.urlencoded():** Ce middleware est utilisé pour parser les données envoyées par les formulaires HTML (encodées en URL).
- **Autres middlewares:** Il existe de nombreux autres middlewares disponibles pour Express.js, chacun ayant une fonction spécifique.

## Utiliser un outil tiers pour requêter une API

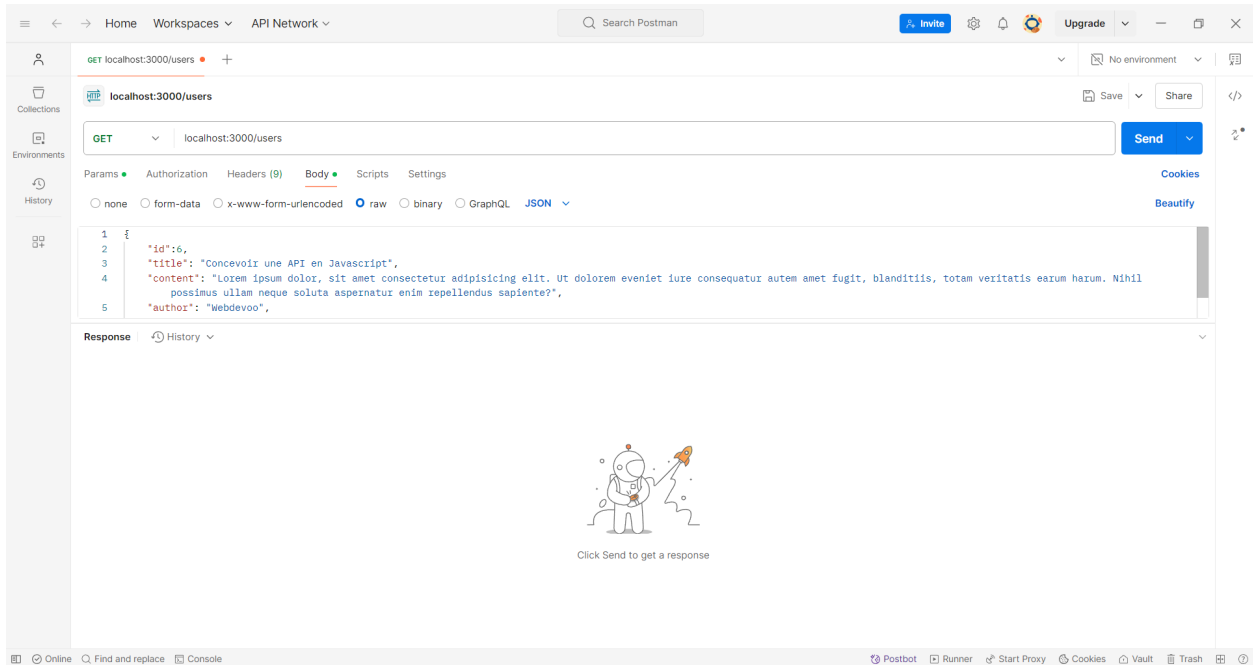
[Postman en RAW(JSON)]

Nous allons utiliser un outil tiers pour requêter facilement notre API.

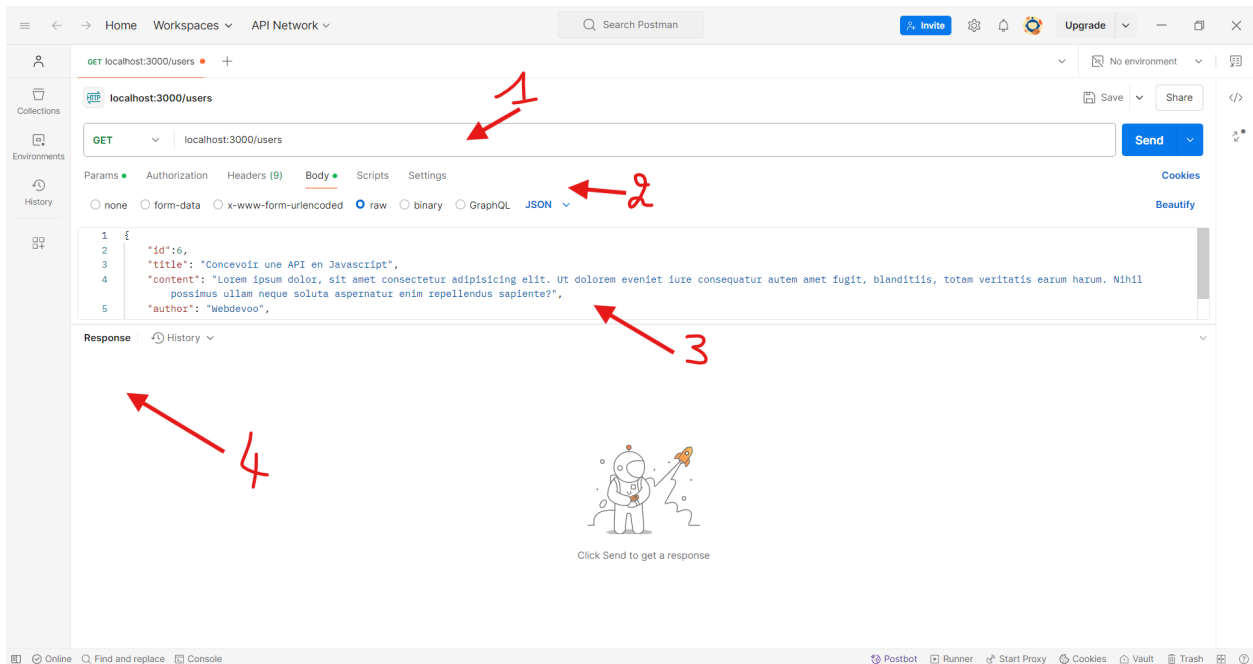
Cet outil s'appelle **POSTMAN** et vous pouvez le télécharger ici :

<https://www.postman.com/downloads/>.

Si vous souhaitez utiliser un outil open-source qui ressemble à Postman, vous pouvez également utiliser **Bruno** : <https://www.usebruno.com/downloads>.



Analysons la fenêtre de Postman :



1. Cette partie correspond à la barre de requête :

- 1.1. **GET** : Ici, une liste déroulante vous permet de sélectionner le type de requête (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS).
- 1.2. **localhost:3000/users** : Dans cet input, vous pouvez ajouter l'url de l'API et l'endpoint que vous souhaitez cibler.
- 1.3. **Send** : Ce bouton vous permet de déclencher la requête. Pensez à vérifier les paramètres ci-dessous, en fonction du type de requête sélectionnée.
2. Cette partie correspond à la configuration de votre requête :
  - 2.1. **Params** : Cet onglet vous permet d'ajouter des **queryParams**, qui seront ajoutés à l'url ciblée.
  - 2.2. **Authorization** : Cet onglet vous permet d'ajouter un jeton de sécurité, si votre API ou l'API publique ciblée requiert une authentification. On retrouve majoritairement des authentifications sous la forme de "**Bearer token**".
  - 2.3. **Headers** : Cet onglet vous permet de gérer les **headers** envoyés avec la requête HTTP.
  - 2.4. **Body** : Cet onglet vous permet d'inclure un **corps de données** à la requête.
    - 2.4.1. **none** : pas de paramètres.
    - 2.4.2. **form-data** : paramètres de formulaires au format **text** ou **file**.
    - 2.4.3. **x-www-form-urlencoded** : paramètres de formulaires avancés.
    - 2.4.4. **raw** : vous permet d'intégrer des données brut, en fonction des types de données proposés.



- 2.4.5. **binary** : vous permet d'inclure un fichier binaire.
- 2.4.6. **GraphQL** : vous permet de requêter une API GraphQL, qui est différente

d'une API REST.

**2.5. Scripts** : Cet onglet vous permet d'intégrer des scripts Javascript, qui s'exécutent avant et après la requête HTTP.

**2.6. Settings** : Cet onglet vous permet de modifier de nombreux paramètres de Postman.

## Utilisation d'un fichier .env pour sécuriser les données d'authentification

Pour ce projet, nous allons devoir utiliser une base de données et configurer un port d'écoute pour notre serveur local.

Nous allons donc utiliser un **fichier d'environnement** pour centraliser ces données.

Grâce à ce fichier d'environnement, nous pourrions également sécuriser les données incluses, en prenant garde à toujours exclure le fichier **".env"** des emplacements où le code source est accessible au public.

**Créez un fichier **".env"** à la racine du projet **"cv-api"**.**

À l'intérieur, préparez ces données :

```
DATABASE_USER=postgres  
  
DATABASE_HOST=localhost  
  
DATABASE_NAME=cv-api  
  
DATABASE_PASSWORD=root  
  
DATABASE_PORT=5432  
  
PORT=3000
```

**Installez la dépendance **"dotenv"**, via le terminal de commande :**

```
npm i dotenv
```

Ajoutez la ligne suivante au début de votre fichier "index.js" :

```
require("dotenv").config();
```

Désormais, vous pourrez appeler les configurations présentes dans votre fichier d'environnement avec l'instruction suivante :

```
process.env.VOTRE_CONFIGURATION,
```

Par exemple :

```
process.env.DATABASE_USER,
```

## Bénéficier du rechargement à chaud (hot reload) grâce à nodemon

Pour la suite du projet, nous allons apporter des modifications aux fichiers.

Pour éviter de relancer constamment la commande "**node index.js**", nous allons installer une dépendance globalement sur la machine, qui se nomme "**nodemon**".

**Nodemon** nous permettra de lancer notre application sur une url et un port spécifique et s'occupera du rechargement à chaud des fichiers à chaque modification (dès la sauvegarde du fichier).

Installez globalement nodemon sur la machine, avec la commande suivante :

```
npm install -g nodemon
```

Lancez le serveur nodemon sur "localhost" et le port "3000".

```
nodemon index.js localhost 3000
```

## Tester la requête POST, avec Postman, pour créer un utilisateur

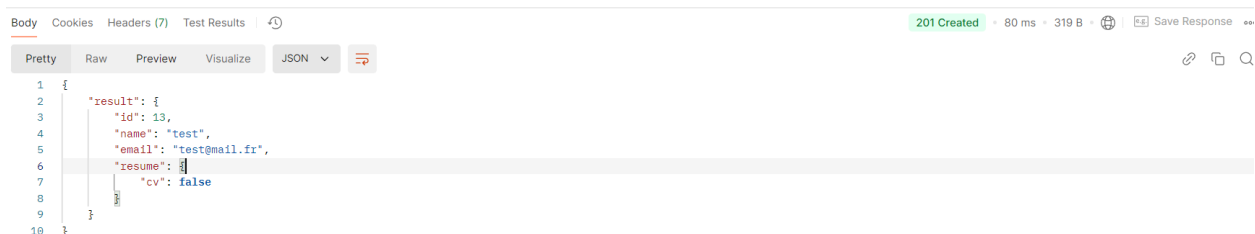
Avec Postman, nous allons créer un nouvel utilisateur.

- Configurez le type de requête sur **POST**.
- Entrez l'url suivante : **localhost:3000/users** (endpoint **POST** que nous avons ajouté au fichier **index.js** au début de la session).
- Allez dans l'onglet **Body** et sélectionnez le type **Raw**, puis spécifiez le format **JSON**.
- Ajoutez les données suivantes :

```
◆ {  
  "name": "test",  
  "email": "test@mail.fr",  
  "resume": {  
    "cv": false  
  }  
}
```

- Cliquez sur le bouton **Send**, pour lancer la requête.

Vous devriez donc avoir un retour de requête au **statut 201** (Created) et cette visualisation de données :



Vous venez d'ajouter votre premier utilisateur via l'API.

Si vous avez une erreur, pensez à vérifier que votre serveur est bien lancé.



## Écouter une requête GET sur l'endpoint `"/users"`

Nous avons initialisé le projet avec un peu de code par défaut, qui contenait notre route `"/users"`, configuré pour écouter les **requêtes POST**.

Nous allons désormais créer l'endpoint `"/users"` pour écouter les requêtes **GET**.

Créez une nouvelle route, avec **app.get**, sur l'endpoint `"/users"`. Cette route déclenchera un callback asynchrone avec les arguments **req** et **res** :

```
app.get("/users", async (req, res) => {  
  
});
```

Nous allons sécuriser le déclenchement du callback, en prévoyant la gestion des erreurs.

Créez un bloc **try/catch**. Le bloc **catch** prendra un argument **err**, qui stockera l'erreur remontée et déclenchera le renvoi d'un **statut 500** et un **json** contenant un paramètre **"message"** et le message d'erreur détecté :

```
try{  
  
}catch(err){  
  
    res.status(500).json({  
  
        "message": `Une erreur s'est produite lors de la tentative de  
récupération des données de la table users : ${err}.`  
  
    });  
  
}
```

Dans le bloc **try**, commencez par créer une constante **“result”**, qui stockera le **retour asynchrone** de la requête **“SELECT \* FROM users ORDER BY id ASC”**.

Pour lancer la requête SQL, nous utiliserons la méthode **“query”**, de notre instance de **“pool”** (fourni par la dépendance **pg**).

```
const result = await pool.query("SELECT * FROM users ORDER BY id ASC");
```

Ensuite, vérifiez conditionnellement que **“result.rows”** contienne un résultat. Dans le cas contraire, générez une nouvelle erreur avec **“throw new Error()”**.

```
if(result.rows.length <= 0 || !result.rows){  
  
    throw new Error("La table users est vide ou inexistante.");  
  
}
```

Pour finir, renvoyez un **statut 200**, avec la méthode **status()** de **“res”**, puis chaînez la méthode **json()** en lui passant en paramètre **“result.rows”**.

```
res.status(200).json(result.rows);
```

Vous devriez désormais avoir la route **app.get(“/users”)** suivante :

```
app.get("/users", async (req, res) => {  
  
    try{  
  
        const result = await pool.query("SELECT * FROM users ORDER BY id  
ASC");  
  
  
  
        if(result.rows.length <= 0 || !result.rows){
```

```

        throw new Error("La table users est vide ou inexistante.");
    }

    res.status(200).json(result.rows);

} catch (err) {

    res.status(500).json({

        "message": `Une erreur s'est produite lors de la tentative de
récupération des données de la table users : ${err}.`

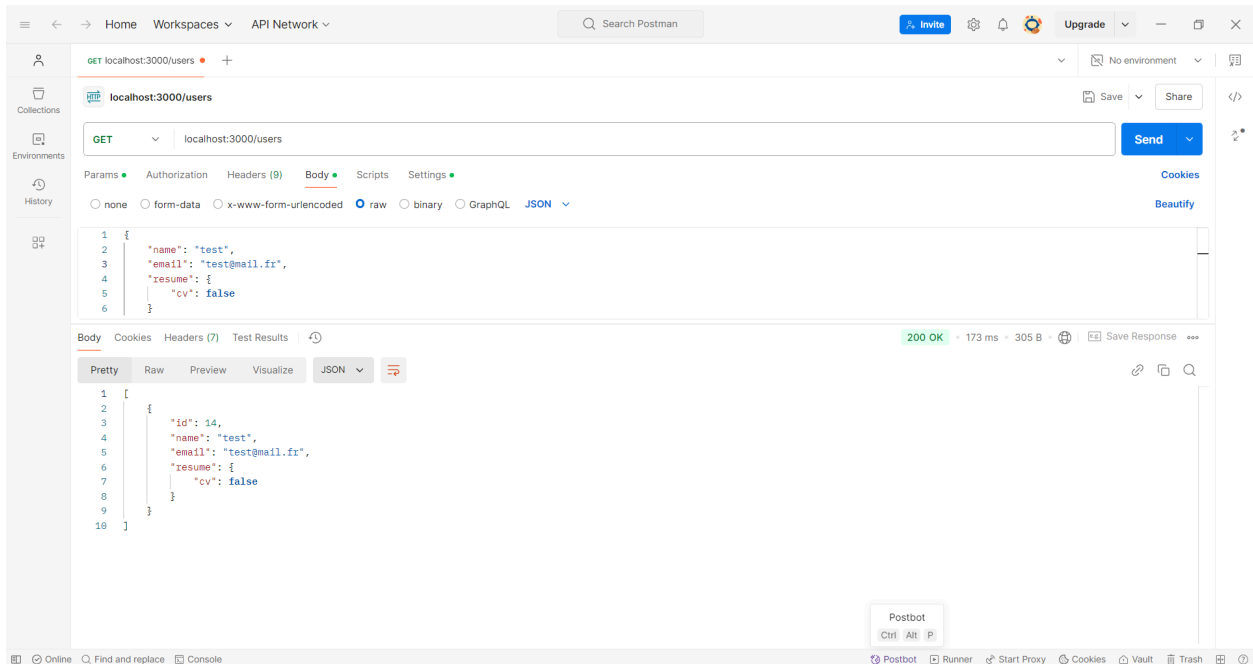
    });

}

});

```

Passez sur **Postman** et lancez une requête **GET** sur la route **"localhost:3000/users"**.



Vous devriez voir apparaître la liste de vos utilisateurs.

## Écouter une requête DELETE sur l'endpoint "/users"

Créez une nouvelle route, sur la méthode **"delete"** de **"app"**, qui ciblera l'endpoint **"/users"** et déclenchera un callback asynchrone prenant en argument **"req"** et **"res"**.

```
app.delete("/users", async (req,res) => {  
  });
```

Créez votre bloc **try/catch**, en initialisant la valeur de votre **catch** en premier.

```
try{  
}catch(err){  
  res.status(500).json({  
    message: `Une erreur s'est produite lors de la suppression d'un  
utilisateur : ${err}`  
  });  
}
```

Dans le bloc **try**, commencez par **décomposer** le paramètre **"id"** qui sera inclut dans les paramètres de requêtes, en utilisant la notation suivante : **"const { id } = req.body"**.

```
const { id } = req.body;
```

Stockez le résultat de la requête **"poll.query("DELETE FROM users WHERE id=\$1", [id])"**, dans une **constante "result"**.

Ici, le paramètre **\$1** va nous permettre de transmettre un **tableau de données** et d'implémenter ces données dynamiquement dans la requête.

Attention, si vous avez plusieurs paramètres, pensez à respecter l'ordre numérique dans votre tableau de données.

```
const result = pool.query("DELETE FROM users WHERE id=$1", [id]);
```

Renvoyez un **statut 200**, avec la méthode **"status()"** de **"res"**, puis chaînez la méthode **"json()"** pour renvoyer les données suivantes :

- **message:** `L'utilisateur n°\${id} a été supprimé.` ,
- **result:** result,

```
res.status(200).json({  
  message: `L'utilisateur n°${id} a été supprimé.` ,  
  result: result  
});
```

Vous devriez désormais avoir la route suivante :

```
app.delete("/users", async (req,res) => {  
  
  try{  
    const { id } = req.body;
```

```

const result = pool.query("DELETE FROM users WHERE id=$1", [id]);
res.status(200).json({
  message: `L'utilisateur n°${id} a été supprimé.`,
  result: result
});
} catch(err) {
  res.status(500).json({
    message: `Une erreur s'est produite lors de la suppression
d'un utilisateur : ${err}`
  });
}
});

```

## Écouter une requête PUT sur l'endpoint "/users"

Créez une nouvelle route, sur la méthode **"put"** de **"app"**, qui ciblera l'endpoint **"/users"** et déclenchera un callback asynchrone prenant en argument **"req"** et **"res"**.

Cette requête peut être utilisée pour deux cas de figures :

- Mettre à jour un objet ENTIER de la table
- Créer une nouvelle entrée dans la table si elle n'est pas existante

```

app.put("/users/edit", async (req, res) => {
});

```

Préparez votre bloc **try/catch**, en initialisant les données du bloc **catch**.

```

try{
} catch(err) {

```

```

    res.status(500).json({
      "message": `Echec de la tentative de mise à jour des données
sur la table users : ${err}`
    });
  }
}

```

Dans le bloc **try**, commencez par déstructurer les paramètres suivants : **"id"**, **"name"**, **"email"**, **"resume"**, depuis le **"req.body"**.

Le paramètre **"id"** doit être **"null"** par défaut.

```

const {id = null, name, email, resume} = req.body;

```

Vérifiez conditionnellement si **"id"** est **"null"**.

Dans ce cas :

- Stockez, dans une **constante "result"** le retour **asynchrone** de la requête **"pool.query("INSERT INTO users(name, email, resume) VALUES(\$1, \$2, \$3) RETURNING \*", [name, email, resume])"**.  
 Cette requête a pour but de créer une nouvelle entrée de données dans la table **users**.
- Renvoyez un **statut 201**, avec la méthode **status()** de **"res"** et chaînez le **json()** suivant :
  - ◆ **message: "Un nouvel utilisateur a été ajouté à la table users."**,
  - ◆ **result: result.rows**

```

if(id === null){
  const result = await pool.query("INSERT INTO users(name, email,
resume) VALUES($1, $2, $3) RETURNING *", [name, email, resume]);
  res.status(201).json({
    message: "Un nouvel utilisateur a été ajouté à la table users.",

```

```

        result: result.rows
    });
} else {
}

```

Dans le cas où **"id"** n'est pas **"null"** :

- Stockez, dans une **constante "result"** le retour **asynchrone** de la requête **"pool.query("UPDATE users SET name=\$2, email=\$3, resume=\$4 WHERE id=\$1", [id, name, email, resume])"**.  
 Cette requête a pour but de modifier un objet de données dans la table users.  
 On utilise **"\$1"** pour le paramètre **"id"**, donc on place **"id"** en premier dans le tableau de données fourni.
- Renvoyez un **statut 201**, avec la méthode **status()** de **"res"** et chaînez le **json()** suivant :
  - ◆ **message:** "Un nouvel utilisateur a été ajouté à la table users.",
  - ◆ **result:** result.rows

```

else{
    const result = await pool.query("UPDATE users SET name=$2, email=$3,
resume=$4 WHERE id=$1", [id, name, email, resume]);
    res.status(200).json(result);
}

```



Vous devriez désormais avoir le code suivant :

```
app.put("/users/edit", async (req, res) => {
  try{
    const {id = null, name, email, resume} = req.body;
    if(id === null){
      if(verifyUnicity(email)){
        throw new Error("L'email est déjà utilisé par un
utilisateur.");
      }
      const result = await pool.query("INSERT INTO users(name, email,
resume) VALUES($1, $2, $3) RETURNING *", [name, email, resume]);
      res.status(201).json({
        message: "Un nouvel utilisateur a été ajouté à la table users.",
        result: result.rows
      });
    }else{
      const result = await pool.query("UPDATE users SET name=$2,
email=$3, resume=$4 WHERE id=$1", [id, name, email, resume]);
      res.status(200).json(result);
    }
  }catch(err){
    res.status(500).json({
      "message": `Echec de la tentative de mise à jour des données
sur la table users : ${err}`
    });
  }
});
```

## Écouter une requête PATCH sur l'endpoint `"/users/edit/name"`

Créez une nouvelle route, sur la méthode **"patch"** de **"app"**, qui ciblera l'endpoint **`"/users/edit/name"`** et déclenchera un callback asynchrone prenant en argument **"req"** et **"res"**.

Cette requête peut être utilisée pour un seul cas de figure :

→ **Mettre à jour PARTIELLEMENT un objet**

Pour respecter les bonnes pratiques, il est conseillé de ne mettre à jour qu'un paramètre par requête **PATCH**, pour **affiner sa logique**.

```
app.patch("/users/edit/name", async (req, res) => {  
  });
```

Préparez votre bloc **try/catch**, en initialisant les données du bloc **catch**.

```
try{  
}catch(err) {  
  res.status(500).json({  
    message: `Une erreur s'est produite lors de la mise à jour du  
nom de l'utilisateur : ${err}`  
  });  
}
```

Dans le bloc **try**, commencez par déstructurer les paramètres suivants : **"id"**, **"name"**, **"...params"** depuis le **"req.body"**.

Le paramètre **"...params"** utilise le **REST operator** pour stocker tous les paramètres supplémentaires dans une **liste de paramètres**.

Pour notre méthode, elle est optionnelle, mais elle vous permettra de voir la structure utilisée pour stocker ces données.

```
const {id, name, ...params} = req.body;
```

Stockez dans une **constante "result"** le retour **asynchrone** de la requête **"pool.query("UPDATE users SET name=\$2 WHERE id=\$1", [id, name])"**.

```
const result = pool.query("UPDATE users SET name=$2 WHERE id=$1", [id, name]);
```

Retournez un **statut 200** avec la méthode **status()** de **res**, puis chaînez le **json()** suivant :

- **message:** "Le nom de l'utilisateur a été modifié",
- **result:** result,
- **params:** params

```
res.status(200).json({  
  message: "Le nom de l'utilisateur a été modifié.",  
  result: result,  
  params: params  
});
```

Vous devriez désormais avoir le code suivant :

```
app.patch("/users/edit/name", async (req, res) => {  
  try{  
    const {id, name, ...params} = req.body;
```

```

    const result = pool.query("UPDATE users SET name=$2 WHERE id=$1",
[id, name]);

    res.status(200).json({
      message: "Le nom de l'utilisateur a été modifié.",
      result: result,
      params: params
    });
  } catch (err) {
    res.status(500).json({
      message: `Une erreur s'est produite lors de la mise à jour du
nom de l'utilisateur : ${err}`
    });
  }
});

```

## Tester la requête PUT sur l'endpoint "/users/edit"

Ouvrez Postman et lancez une requête **GET** sur l'endpoint **"/users"**.

Récupérez les **données d'un utilisateur**, et copiez les.

Par exemple :

```

{
  "id": 14,
  "name": "test",
  "email": "test@mail.fr",
  "resume": {
    "cv": false
  }
}

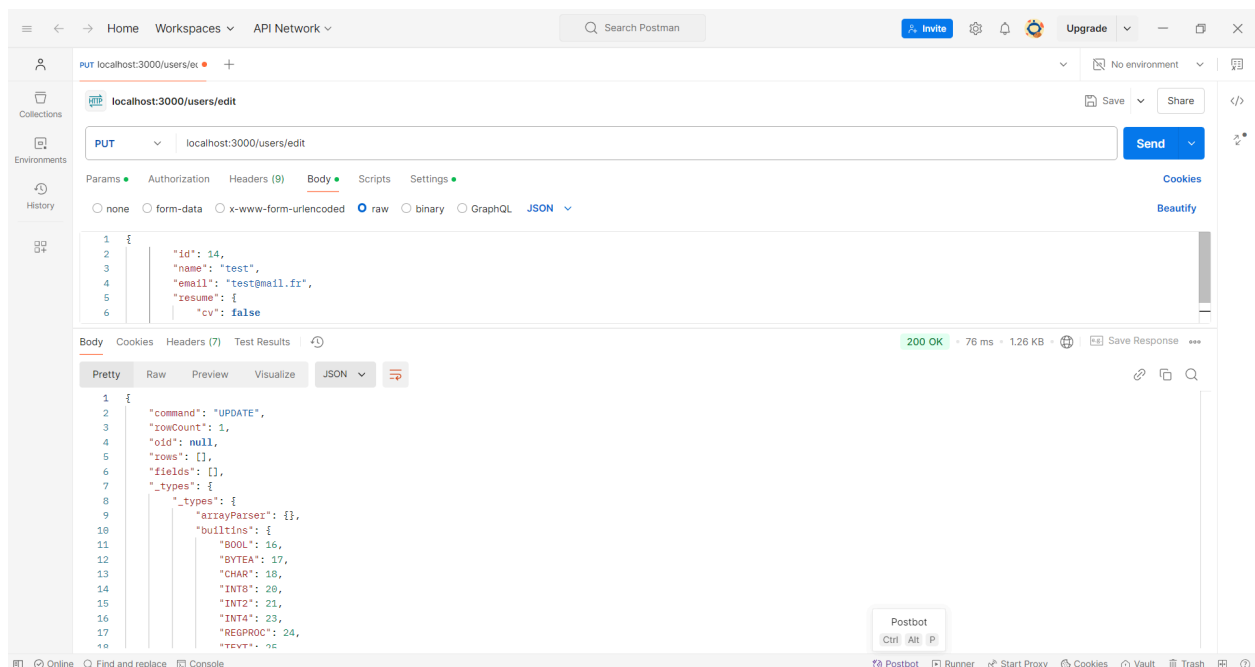
```

Changez le type de requête en **PUT** et modifiez l'endpoint pour cibler **"/users/edit"**.

Collez les données récupérées dans l'onglet **Body > RAW (JSON)** et modifiez les paramètres **"name"** et **"email"** des données récupérées.

```
{  
  "id": 14,  
  "name": "tester",  
  "email": "tester@mail.fr",  
  "resume": {  
    "cv": false  
  }  
}
```

Ensuite, envoyez la requête.



Si votre requête se passe correctement, vous devriez voir votre retour de **"result.rows"**, dans

la visualisation de données.

Ce retour est important pour le projet **cv-api**, car il vous permet de connaître les valeurs renvoyées par **"pool.query"**.

## Tester la requête PATCH sur l'endpoint **"/users/edit/name"**

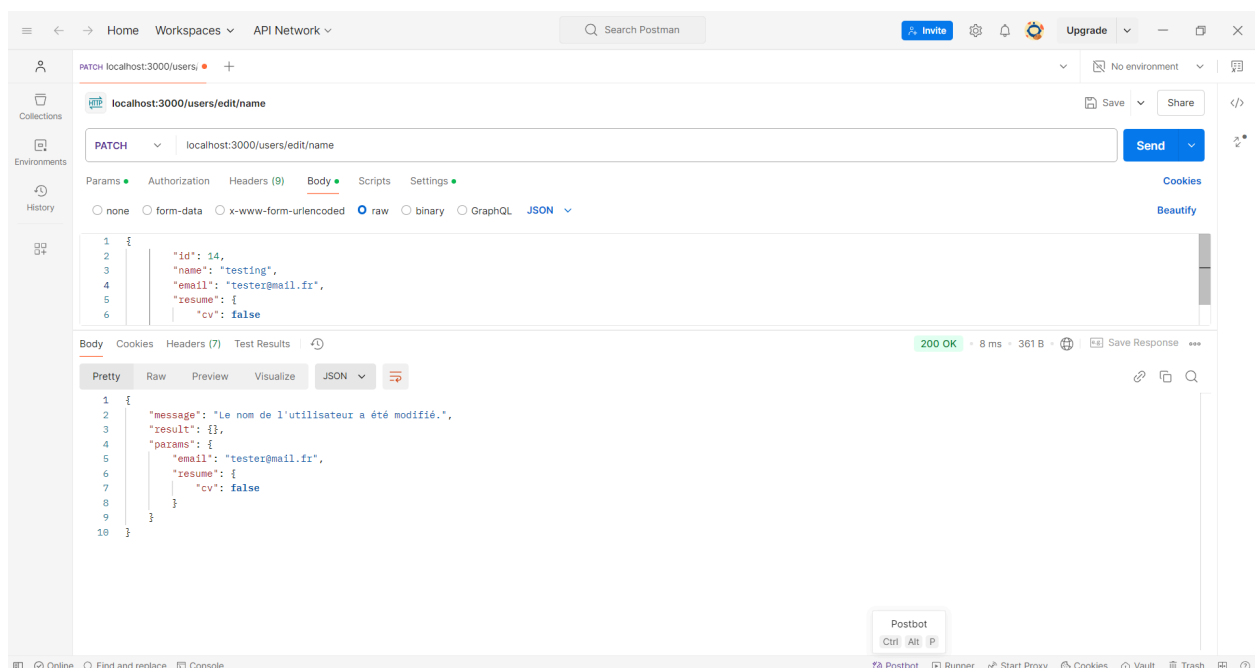
En conservant les données de la requête **PUT**, modifiez simplement le paramètre **"name"** des données fournies et conservez uniquement les paramètres **"id"** et **"name"**.

Par exemple :

```
{  
  "id": 14,  
  "name": "tester",  
}
```

Modifiez le type de requête pour passer en mode **PATCH**.

Modifiez l'endpoint pour **"/users/edit/name"** et lancez la requête.



## Exercices autonomes

1. Créez les requêtes **PATCH** pour les autres paramètres de la table **users**.
2. Créez une méthode de vérification d'unicité d'un paramètre particulier et utilisez le sur les requêtes de création et de modification de données.

## Entraînement avec POSTMAN

Pour vous entraîner à **requêter des API avec Postman**, vous pouvez utiliser **JSON Placeholder**, qui est une API publique ne nécessitant pas d'authentification particulière.

<https://jsonplaceholder.typicode.com/>